

2.2. Effacer ce qu'on a écrit

Il n'y a pas de gomme : on écrit du vide par-dessus.

1. Ce qu'il faut comprendre

La carte de fond garde ce qu'on y a mis, tant qu'on ne le remplace pas. **L'espace est la tuile 0**, c'est-à-dire du vide : écrire des espaces par-dessus **est** l'effacement. C'est ce que fait ce programme, et c'est ce qu'il faut avoir compris une fois.

Le piège tient en un mot : il faut **autant d'espaces que de lettres**. `BONJOUR` en compte sept ; six espaces laisseraient un `R` tout seul au bout, et cette lettre orpheline est l'erreur la plus courante du débutant.

C'est pour cela que `effacer(6, 4, "BONJOUR")` **existe** : il ôte exactement la longueur du texte, comptée par le compilateur. Les deux lignes font la même chose, mais l'une ne peut pas se tromper d'un espace. La leçon « Effacer sans compter les lettres » y revient.

Pour effacer une zone plus large, `poser(colonne, ligne, 0)` fait la même chose case par case, et se met dans une boucle.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **605 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         90 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
LireManette    37 o ← bouton() - lit les huit boutons
Tuiles         208 o ← CopierTuiles - 13 lettres sur 43
"BONJOUR"      7 o ← texte() - un texte gravé
"APPUIE SUR A" 12 o ← texte() - un texte gravé
"      "       7 o ← texte() - un texte gravé
"MERCI      "  12 o ← texte() - un texte gravé
"BONJOUR"      7 o ← texte() - un texte gravé
"APPUIE SUR A" 12 o ← texte() - un texte gravé
```

Lettres gravées : ABCEIJMNOPRSU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,

DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 6	66 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 2	10 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>    // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(6, 4, "BONJOUR");
    texte(3, 8, "APPUIE SUR A");

    while (true) {
        image();

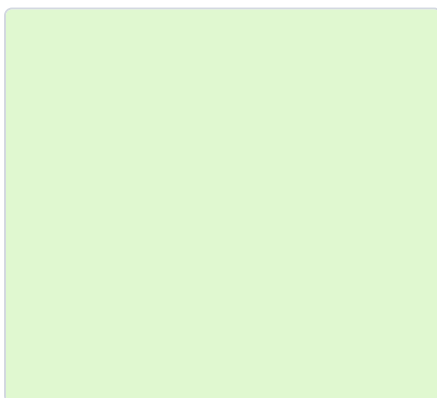
        if (bouton(A)) {
            texte(6, 4, "      "); // sept espaces, autant que BONJOUR
            texte(3, 8, "MERCI      ");
        }

        if (bouton(B)) {
            texte(6, 4, "BONJOUR");
            texte(3, 8, "APPUIE SUR A");
        }
    }

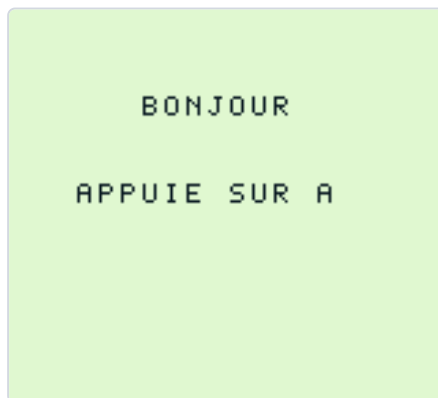
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

MERCI

A — l'écran a changé

BONJOUR

APPUIE SUR A

B — l'écran a changé

5. Ce qu'on doit voir

A efface « BONJOUR » ; B le remet.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ « BONJOUR » est là au départ

✓ A l'efface entièrement

✓ A écrit « MERCI »

✓ B le remet

✓ et la consigne aussi

Ensuite : **2.3. Les messages dans leur propre fichier** — chapitre 1 : Les tout premiers pas. · gameboy3 — écrire une cartouche Game Boy en C++